

תוכנית מקצועית להקמת מערכת סינון אנדרואיד

מטרה

לבנות מערכת סינון לאנדרואיד שמתחילה כפילוט טכני פשוט, ומתקדמת בהדרגה למוצר חזק ברמת ניהול מכשיר מלא.

המטרה אינה לבנות "אפליקציית VPN רגילה", אלא מערכת שמנהלת את המכשיר, את האפליקציות, את הגלישה, ואת מדיניות ההרשאות בצורה מסודרת.

עיקרון בסיסי

VPN בלבד = התחלה טובה, אבל לא מוצר חזק

אפליקציית VPN יכולה לסנן גלישה, דומיינים ותעבורה, אבל היא לא מספיקה כדי לשלוט באמת במכשיר.

Device Owner / Android Enterprise = הכיוון הרציני

מוצר חזק צריך להגיע בסוף למצב שבו המכשיר מנוהל, עם:

- Always-on VPN
- Lockdown אם ה־VPN נפל
- חסימת הסרה
- שליטה באפליקציות
- חסימת התקנות לא מאושרות
- Launcher סגור
- allowlist לפי חתימת אפליקציה
- פאנל ניהול מרחוק

שלב 0 – הגדרת מודל הגנה

משך מומלץ: שבוע

לפני שכותבים קוד, צריך להגדיר בדיוק מול מה המערכת מגינה.

שאלות שצריך לענות עליהן

1. האם המכשיר שייך למשתמש או נמכר כמכשיר מנוהל?
2. האם המשתמש הוא אדם רגיל או משתמש טכני שמנסה לעקוף?
3. האם מותר לו להתקין אפליקציות לבד?
4. האם מותר לו להיכנס להגדרות?
5. האם מותר לו לכבות VPN?
6. האם מותר לו להתקין APK מחוץ ל־Google Play?
7. האם רוצים לחסום רק דפדפן, או גם אפליקציות פנימיות עם WebView?

8. האם רוצים סינון לפי אתר, לפי אפליקציה, או לפי שניהם?
9. האם המערכת אמורה לעבוד גם בלי אינטרנט לכמה דקות?
10. מה קורה אם השרת לא זמין?

החלטה מומלצת

המדיניות צריכה להיות:

Default deny.

כלומר:

- מה שמאושר — פתוח.
- מה שלא מאושר — חסום.
- אם יש ספק — חסום.
- אם השרת לא זמין — עובדים לפי policy cache מקומי.
- אם אין policy מקומי — חוסמים גלישה, אבל משאירים ערוץ תמיכה/חיבור מינימלי.

שלב 1 — MVP טכני ראשון

משך מומלץ: 2-4 שבועות

המטרה של השלב הזה היא להוכיח שיש לך שליטה בסיסית בגלישה ובאפליקציות.

רכיבי MVP

1. אפליקציית Android Agent

טכנולוגיה מומלצת:

- Kotlin
- Native Android
- Foreground Service
- VpnService
- Room DB לשמירת מדיניות מקומית
- WorkManager למשימות רקע
- Usage Access לזיהוי אפליקציה פתוחה
- Overlay למסך חסימה, רק איפה שנדרש

יכולות ראשונות

- הפעלת VPN מקומי
- העברת תעבורה דרך מנגנון סינון
- חסימת דומיינים בסיסית
- allowlist / blacklist
- זיהוי האפליקציה הפעילה
- הצגת מסך חסימה

- שמירת policy מקומי
- מסך סטטוס: מוגן / לא מוגן / שגיאה
- כפתור "בקש פתיחה"

לא לבנות בשלב הזה

- לא TLS inspection מלא
- לא סינון תמונות מתקדם
- לא AI classification בזמן אמת
- לא תמיכה ב־iPhone
- לא Marketplace
- לא מערכת רבנים/קהילות
- לא לנסות לפתור את כל סוגי העקיפה ביום הראשון

המטרה היא להגיע למכשיר אחד שעובד טוב, לא למוצר מושלם.

שלב 2 – שרת ניהול בסיסי

משך מומלץ: 2-4 שבועות

כאן הופכים את האפליקציה ממוצר מקומי למערכת מנוהלת.

Backend מומלץ

אפשרות א':

- FastAPI
- PostgreSQL
- Redis
- Docker

אפשרות ב':

- NestJS
- PostgreSQL
- Redis
- Docker

אם אתה כבר חזק ב־Python, הייתי מתחיל עם FastAPI.

טבלאות בסיסיות

users

- id
- name
- phone
- role

status •

devices

id •
user_id •
device_name •
android_version •
manufacturer •
model •
agent_version •
last_seen •
protection_status •

policies

id •
name •
description •
default_action •
created_at •

policy_rules

id •
policy_id •
rule_type •
value •
action •
priority •

דוגמאות ל־rule_type:

domain •
package •
app_signature •
category •
time_limit •
url_pattern •

apps

id •
package_name •
app_name •
signing_cert_sha256 •
installer •
version_code •
status •

requests

id •

- device_id •
- request_type •
- target •
- reason •
- status •
- admin_note •
- created_at •

events

- id •
- device_id •
- event_type •
- details •
- created_at •

API בסיסי

Device registration

POST /api/devices/register

Heartbeat

POST /api/devices/heartbeat

Pull policy

GET /api/devices/{id}/policy

Report event

POST /api/devices/{id}/events

Create opening request

POST /api/requests

Admin approve/reject

POST /api/requests/{id}/approve

POST /api/requests/{id}/reject

שלב 3 — מנוע סינון

משך מומלץ: 3-6 שבועות

כאן בונים את הליבה של הסינון.

התחלה מומלצת

להתחיל עם:

- DNS filtering
- domain allowlist
- domain blacklist
- SNI filtering איפה שאפשר
- per-app policy
- block page
- policy cache מקומי

למה לא להתחיל מ־TLS inspection

TLS inspection יכול לתת יותר שליטה, אבל הוא יוצר בעיות:

- אפליקציות בנק עלולות להישבר
- Google Pay עלול להישבר
- אפליקציות תעופה עלולות להישבר
- אפליקציות עם certificate pinning לא יעבדו
- יש סיכוני פרטיות גדולים
- צריך להתקין תעודת CA
- זה מסבך מאוד תמיכה

החלטה מומלצת

בגרסה הראשונה:

לא עושים MITM מלא.

עושים:

- DNS
- SNI
- IP/domain categories
- per-app rules
- app allowlist
- block screen ברור

רק בגרסה מתקדמת בודקים TLS inspection, וגם אז רק למסלולים מסוימים ולא לכל האפליקציות.

שלב 4 — זיהוי אפליקציות נכון

משך מומלץ: 2-4 שבועות

זה שלב קריטי.

אסור לסמוך רק על package name.

לא נכון

לאשר אפליקציה רק לפי:

com.whatsapp

נכון

לאשר אפליקציה לפי שילוב של:

- package name
- signing certificate SHA-256
- installer source
- version code
- known official app
- האם הותקנה מ־Managed Play / Play Store
- האם היא dual app / clone
- האם היא בפרופיל אחר
- האם היא APK חיצוני

כלל מומלץ

אפליקציה מותרת רק אם:

1. שם החבילה תואם
2. החתימה הקריפטוגרפית תואמת
3. המקור תקין
4. הגרסה לא חשודה
5. המדיניות מאפשרת אותה

אם אחד מהם לא תקין — חסום.

שלב 5 — הגנות נגד כיבוי ושיבוש

משך מומלץ: 3-5 שבועות

בשלב הזה מחזקים את המוצר.

דברים שצריך לזהות

- VPN כבוי
- הרשאת Usage Access בוטלה
- הרשאת Overlay בוטלה
- Accessibility בוטל
- האפליקציה הוסרה
- האפליקציה force stopped
- מצב טיסה הופעל

- DNS פרטי הופעל
- הותקן VPN אחר
- הותקן APK חדש
- הופעל Safe Mode
- נעשה factory reset
- הותקן clone app
- נפתח Secure Folder / Dual Apps

תגובה מומלצת

לא לפעול בצורה אגרסיבית או חשודה.

כן לעשות:

- לוג אירוע
- שליחת alert לשרת
- חזרה אוטומטית למצב מוגן כשאפשר
- הצגת מסך "ההגנה לא פעילה"
- חסימת גלישה אם אין VPN
- דרישת קוד מנהל לשינוי מצב

שלב 6 — מעבר ל-Android / Device Owner Enterprise

משך מומלץ: 4-8 שבועות

זה השלב שהופך את המוצר מרמה בינונית לרמה רצינית.

למה זה חשוב

Device Owner נותן למערכת יכולות שלא קיימות לאפליקציה רגילה.

יכולות שצריך לשאוף אליהן

- Always-on VPN
- VPN lockdown
- חסימת תעבורה כשה־VPN נפל
- מניעת הסרת agent
- חסימת התקנת אפליקציות לא מאושרות
- חסימת מקורות לא ידועים
- הגבלת הגדרות
- ניהול תעודות
- ניהול אפליקציות
- Launcher סגור
- Lock Task Mode
- הגדרת פרופילים מרחוק

שתי אפשרויות טכניות

אפשרות א' — Android Management API

יתרונות:

- מהיר יותר
- לא צריך לבנות DPC מלא מהתחלה
- מתאים לפיילוט
- משתמש בתשתית Android Enterprise של Google
- טוב למכשירים fully managed / dedicated

חסרונות:

- תלות ב-Google
- פחות שליטה עמוקה במנגנונים פנימיים
- צריך להבין היטב את מודל המדיניות שלהם

אפשרות ב' — DPC עצמאי

יתרונות:

- שליטה מלאה
- מותאם בדיוק לצרכים שלך
- טוב לטווח ארוך

חסרונות:

- מורכב מאוד
- דורש הרבה ניסיון Android Enterprise
- יותר קשה לתחזק
- יותר סיכון לבאגים

החלטה מומלצת

להתחיל עם Android Management API.

רק אם המוצר מוכיח את עצמו ויש צורך אמיתי — לבנות DPC עצמאי.

שלב 7 — Launcher סגור

משך מומלץ: 3-5 שבועות

במקום לרדוף אחרי כל אפליקציה, עדיף לשלוט במסך הבית.

מה ה-Launcher צריך לכלול

- רשימת אפליקציות מאושרות בלבד

- חיפוש אפליקציות מאושרות
- כפתור בקשת אפליקציה
- כפתור תמיכה
- סטטוס הגנה
- הודעות מערכת
- מצב "עסק"
- מצב "נסיעה"
- מצב "בית"
- קוד מנהל לפתיחת הגדרות

יתרון

משתמש רגיל לא צריך להרגיש שהוא "נחסם כל הזמן".

הוא פשוט רואה סביבת עבודה נקייה עם מה שמותר לו.

שלב 8 — פאנל ניהול

משך מומלץ: 3-6 שבועות

טכנולוגיה מומלצת

- Next.js
- TypeScript
- Tailwind
- PostgreSQL
- Auth עם 2FA
- Role-based access

מסכים חשובים

Dashboard

- כמה מכשירים פעילים
- כמה מכשירים לא מוגנים
- כמה חסימות היום
- כמה בקשות פתיחה מחכות
- בעיות מערכת

Devices

- שם מכשיר
- משתמש
- גרסת Android
- סטטוס VPN
- סטטוס הרשאות
- גרסת Agent

- last seen •
- policy assigned •

Policies

- policy יצירת •
- allowlist עריכת •
- blacklist עריכת •
- הגדרת אפליקציות •
- הגדרת שעות •
- הגדרת חריגים •

Requests

- בקשות לפתיחת אתר •
- בקשות לפתיחת אפליקציה •
- אישור זמני •
- אישור קבוע •
- דחייה עם סיבה •

Logs

- ניסיונות חסימה •
- שינוי הרשאות •
- אפליקציות חדשות •
- תקלות VPN •
- אירועי אבטחה •

שלב 9 — מערכת בקשות פתיחה

משך מומלץ: 2-4 שבועות

זה יכול להיות היתרון הכי גדול מול מתחרים.

זרימה מומלצת

1. המשתמש מנסה לפתוח אתר או אפליקציה.
2. המערכת חוסמת.
3. מופיע כפתור "בקש פתיחה".
4. המשתמש כותב סיבה.
5. הבקשה מגיעה לפאנל.
6. מנהל מאשר או דוחה.
7. המכשיר מקבל policy מעודכן.
8. המשתמש מקבל הודעה.

סוגי אישור

- אישור ל-15 דקות

- אישור לשעה
- אישור ליום
- אישור קבוע
- אישור רק לאפליקציה מסוימת
- אישור רק למכשיר מסוים
- אישור רק בשעות מסוימות

בהמשך

אפשר להוסיף WhatsApp bot:

- שליחת בקשה
- בדיקת סטטוס
- קבלת אישור
- הודעה למנהל
- קישור לפאנל

שלב 10 — QA ובדיקות עקיפה מבוקרות

משך מומלץ: מתמשך

צריך לבנות רשימת בדיקות קבועה.

מכשירי בדיקה

לפחות:

- Samsung זול
- Samsung בינוני
- Xiaomi / Redmi
- Pixel
- Android ישן יחסית
- Android חדש
- טאבלט

בדיקות חובה

- הפעלת VPN
- נפילת VPN
- reboot
- כיבוי הרשאות
- הסרת אפליקציה
- התקנת APK
- התקנת VPN אחר
- שינוי DNS
- פתיחת WebView
- פתיחת דפדפן פנימי

- Google Play
- Waze
- WhatsApp
- Gmail
- בנקים
- Google Pay
- אפליקציות תעופה
- מצב חו"ל / roaming
- רשת Wi-Fi ציבורית
- hotspot

מדד הצלחה

המוצר לא נמדד רק לפי "חוסם או לא".

הוא נמדד לפי:

- האם המשתמש מבין מה קרה
- האם יש דרך לבקש פתיחה
- האם המנהל רואה לוג ברור
- האם המכשיר חוזר למצב מוגן
- האם אין יותר מדי false positives
- האם אפליקציות חשובות לא נשברות

שלב 11 – פרטיות ואמון

גם אם כרגע אתה חושב טכני בלבד, זה חייב להיכנס לארכיטקטורה מהיום הראשון.

לאסוף מינימום מידע

לא לשמור:

- תוכן הודעות
- תמונות אישיות
- סיסמאות
- תוכן בנקאי
- טקסטים פרטיים

כן לשמור:

- דומיין שנחסם
- אפליקציה שנפתחה
- סיבת חסימה
- סטטוס הגנה
- בקשות פתיחה
- אירועי אבטחה

כלל חשוב

המערכת צריכה להיות "סינן", לא "ריגול".

סטאק מומלץ

Android

- Kotlin •
- Native Android •
- VpnService •
- DevicePolicyManager •
- UsageStatsManager •
- WorkManager •
- Room •
- Foreground Service •
- Jetpack Compose או XML רגיל
- Firebase Cloud Messaging לעדכוני policy

Backend

המלצה אישית:

- Python FastAPI •
- PostgreSQL •
- Redis •
- SQLAlchemy •
- Alembic •
- Docker •
- Celery / RQ לתורים

או:

- Node.js NestJS •
- PostgreSQL •
- Redis •
- Prisma •
- Docker •

Admin Panel

- Next.js •
- TypeScript •
- Tailwind •
- shadcn/ui •
- NextAuth / Auth.js •
- Role-based permissions •

Hosting

לפיילוט:

Railway / Render / VPS •

למוצר רציני:

Hetzner / DigitalOcean / AWS / GCP •

PostgreSQL מנוהל •

Redis מנוהל •

Monitoring •

Backups •

Roadmap מומלץ ל-90 יום

ימים 1-7 — תכנון ומעבדה

יעדים:

- לבחור 3 מכשירי בדיקה
- להגדיר threat model
- להחליט MVP
- לפתוח repo
- להכין backend skeleton
- להכין Android skeleton
- להגדיר JSON policy בסיסי

תוצר בסוף השבוע:

- מסמך PRD
- אפליקציה ריקה שרצה
- שרת בסיסי
- מבנה database ראשוני

ימים 8-21 — Android VPN MVP

יעדים:

- להפעיל VpnService
- להעביר DNS דרך הסינון
- לחסום דומיינים
- להציג מסך חסימה
- לשמור policy מקומי
- לשלוח heartbeat

תוצר:

- מכשיר אחד עם סינון בסיסי עובד

ימים 22-35 — פאנל ניהול ראשון

יעדים:

- login למנהל
- רשימת מכשירים
- עריכת policy
- צפייה בלוגים
- בקשות פתיחה בסיסיות

תוצר:

- Admin panel בסיסי שמנהל מכשיר אמיתי

ימים 36-50 — App Control

יעדים:

- זיהוי אפליקציה פתוחה
- חסימת אפליקציות לא מאושרות
- בדיקת package name
- בדיקת signing certificate
- זיהוי התקנת אפליקציה חדשה
- לוגים של אפליקציות

תוצר:

- שליטה בסיסית באפליקציות, לא רק באתרים

ימים 51-65 — Hardening

יעדים:

- זיהוי VPN כבוי
- זיהוי הרשאות שבוטלו
- cache מדיניות חזק
- fail closed
- alerts
- שיפור יציבות
- בדיקות reboot
- בדיקות רשתות שונות

תוצר:

- מערכת שכבר אפשר לתת לפיילוט קטן

ימים 66-80 — Android Enterprise / Device Owner POC

יעדים:

- ללמוד Android Management API
- להגדיר מכשיר fully managed
- להפעיל always-on VPN
- להפעיל lockdown
- לבדוק app allowlist
- לבדוק Lock Task Mode
- לבדוק Launcher סגור

תוצר:

- מכשיר אחד מנוהל ברמת Device Owner

ימים 81-90 — פיילוט סגור

יעדים:

- 5-10 מכשירים
- משתמשים אמיתיים
- מעקב חסימות
- תיקון false positives
- שיפור בקשות פתיחה
- איסוף בעיות
- בניית רשימת פיצ'רים לגרסה 1.0

תוצר:

- החלטה האם המוצר יכול להפוך לגרסה מסחרית

סדר עדיפויות ברור

לבנות קודם

1. Android Agent
2. VPN בסיסי
3. DNS/domain filtering
4. policy cache
5. backend policy

- app detection .6
- app signature verification .7
- admin panel .8
- requests workflow .9
- hardening בסיסי .10

לבנות אחר כך

- Device Owner .1
- Android Management API .2
- Launcher סגור .3
- Lock Task Mode .4
- zero-touch provisioning .5
- per-app VPN מתקדם .6
- AI לסיווג בקשות .7
- מערכת תמיכה .8
- billing .9
- white-label .10

לא לבנות בהתחלה

- iOS .1
- TLS MITM מלא .2
- סינון תמונות בזמן אמת .3
- ROM מותאם .4
- Marketplace .5
- מערכת קהילות .6
- מערכת רבנים .7
- תמיכה בעשרות דגמי מכשירים .8
- פיצ'רים של בקרת הורים מורכבת .9
- מערכת תשלומים .10

הגרסה הכי נכונה להתחלה

גרסה 0.1

מכשיר Android אחד.

יכולות:

- VPN פעיל
- חסימת דומיינים
- רשימת אתרים מאושרים
- זיהוי אפליקציה פתוחה
- חסימת אפליקציה לא מאושרת
- מסך חסימה
- פאנל ניהול בסיסי

• בקשת פתיחה

אם זה עובד טוב — להמשיך.

אם זה לא יציב — לא להוסיף עוד פיצ'רים.

הגדרת הצלחה לגרסה 0.1

הגרסה נחשבת הצלחה אם:

1. המכשיר עובד יום שלם בלי שהסינון נופל.
2. אתרים חסומים באמת נחסמים.
3. אתרים מאושרים נפתחים בלי בעיות.
4. אפליקציות לא מאושרות נחסמות.
5. הפאנל רואה את המכשיר.
6. אפשר לעדכן policy מרחוק.
7. יש לוג ברור למה משהו נחסם.
8. המשתמש יכול לשלוח בקשת פתיחה.
9. אחרי reboot המערכת חוזרת לעבוד.
10. אין פגיעה רצינית בביצועי הטלפון.

החלטות טכניות מומלצות

Backend

להתחיל עם FastAPI.

סיבה:

- מהיר לפיתוח
- מתאים ל-API
- קל לבנות policy engine
- אתה כבר בכיוון Python
- קל לחבר AI בהמשך

Android

להתחיל עם Kotlin Native.

לא React Native.

סיבה:

- צריך גישה עמוקה ל-APIs Android
- VPN / DevicePolicy / Usage Access עובדים טוב יותר native
- פחות בעיות תאימות

• יותר שליטה

Filtering

להתחיל עם DNS + domain.

לא MITM.

סיבה:

- פחות שבירות
- פחות בעיות פרטיות
- יותר קל לדבג
- מספיק ל-MVP

Device Owner

לא ביום הראשון, אבל כן בתוך 90 יום.

סיבה:

- בלי זה המוצר יישאר ברמת אפליקציה רגילה
- עם זה אפשר להגיע לרמת סינון רצינית

ארכיטקטורה כללית

-- | Android Device | |-- Android Agent | |-- VPN Service | |-- App Monitor | |-- Local Policy Cache
Block Screen | |-- Heartbeat Client | |-- Local Filtering | |-- DNS rules | |-- Domain rules | |-- App rules
Backend API | |-- Device Management | |-- Policy Engine | |-- Logs | |-- Requests | |-- Admin -- |
Users | |-- Admin Panel | |-- Devices | |-- Policies | |-- Requests | |-- Logs | |-- Alerts

מבנה Policy לדוגמה

```
, "policy_id": "basic_kosher_001", "default_network_action": "block", "default_app_action": "block" }  
-allowed_domains": [ "google.com", "gmail.com", "waze.com" ], "blocked_domains": [ "example-bad"  
  "site.com" ], "allowed_apps": [ { "package_name": "com.whatsapp", "signature_sha256  
  : "OFFICIAL_SIGNATURE_HERE", "action": "allow" } ], "blocked_apps": [ { "package_name"  
  , "com.android.chrome", "action": "block" } ], "support": { "allow_opening_requests": true"  
    { "temporary_opening_minutes": 30 }
```

MVP מדויק לביצוע ראשון

שבוע ראשון

- לפתוח GitHub repo
- ליצור Android app ריק
- ליצור FastAPI server
- ליצור PostgreSQL schema
- ליצור endpoint לרישום מכשיר
- ליצור endpoint לקבלת policy
- ליצור JSON policy ידני
- להציג באפליקציה סטטוס policy

שבוע שני

- להפעיל VpnService
- לתפוס DNS
- לחסום דומיין אחד
- לאפשר דומיין אחד
- להציג מסך חסימה
- לשלוח log לשרת

שבוע שלישי

- להוסיף Usage Access
- לזהות foreground app
- לחסום אפליקציה לפי package
- לשלוח event לשרת
- להציג בפאנל לוגים

שבוע רביעי

- להוסיף פאנל ניהול
- לערוך allowlist
- לערוך blacklist
- לשלוח בקשת פתיחה
- לאשר בקשה
- לעדכן policy במכשיר

כלל זהב

לא לרדוף אחרי עקיפות אחת־אחת.

לבנות שכבות:

1. סינון תעבורה

2. סינון אפליקציות
3. אימות חתימות
4. cache מקומי
5. alerts
6. Device Owner
7. Launcher סגור
8. provisioning מסודר

מי שבונה רק חסימות נקודתיות — תמיד יפסיד.

מי שבונה מערכת ניהול מכשיר — יכול לנצח.

מסקנה

הדרך הנכונה היא לא להתחיל ממערכת ענקית.

הדרך הנכונה:

1. לבנות Android Agent פשוט.
2. להוסיף VPN filtering.
3. להוסיף App Control.
4. להוסיף Backend ופאנל.
5. להוסיף בקשות פתיחה.
6. לחזק נגד כיבוי ושיבוש.
7. לעבור ל-Device Owner.
8. לבנות Launcher סגור.
9. להריץ פיילוט.
10. רק אז לחשוב על מוצר מסחרי.

היעד הראשון שלך:

תוך 30 יום — מכשיר אחד שמסונן דרך המערכת שלך, עם פאנל ניהול בסיסי.

היעד השני:

תוך 90 יום — מכשיר מנוהל ברמת Device Owner עם Always-on VPN, חסימת אפליקציות, ו-Launcher סגור.

זה המסלול המקצועי.